CREDISHIELD AI

LOAN APPROVAL PREDICTION SYSTEM

A Project Report

Report Detail	Information
Submitted by	Shivang Singh
Program	AI/ML
Institution	IIHMF MNNIT
Project Guide	Mr. Subhash Chaganti
Academic year	2026

Machine Learning • Predictive Analytics • Flask Deployment

Abstract

Loan approval is a high-volume decision problem in which lenders must balance customer service, operational speed and credit risk. This project develops CrediShield AI, an end-to-end machine learning system that predicts whether an application is likely to be approved or rejected from applicant, credit and asset information. The implementation covers data cleaning, outlier treatment, categorical encoding, stratified train–test splitting, standardisation, multi-model training, evaluation, model persistence and deployment through a Flask web application.

Four supervised classification algorithms—Logistic Regression, Decision Tree, Random Forest and XGBoost—were evaluated on a holdout test set. Model selection was based on F1-score so that both precision and recall were considered. Random Forest achieved the highest F1-score (97.80%) and an accuracy of 98.36%, and was therefore stored as the production model. The deployed application accepts eleven input features, applies the same fitted preprocessing transformation used during training, and returns the predicted class together with confidence and class probabilities.

The report describes the problem context, relevant machine learning approaches, dataset and preprocessing choices, system architecture, exploratory and feature-engineering considerations, model development, implementation, and comparative results. It also discusses limitations: the model is a decision-support prototype rather than a substitute for regulated underwriting; fairness, drift, explainability and external validation must be addressed before real lending use.

Project at a Glance

Item	Project Value
Dataset	Loan Approval Prediction Dataset
Observations	4,269 records
Candidate models	4
Selected model	Random Forest
Test accuracy	98.36%
Test F1-score	97.80%
Deployment	Flask web application and JSON API

Table of Contents

List of Figures

List of Tables

Abbreviations

Term	Meaning
API	Application Programming Interface
CIBIL	Credit Information Bureau (India) Limited
EDA	Exploratory Data Analysis
F1	Harmonic mean of precision and recall

IQR	Interquartile Range
ML	Machine Learning
ROC-AUC	Area under the Receiver Operating Characteristic curve

Chapter 1: Introduction

1.1 Background

Financial institutions receive large numbers of loan applications containing demographic, financial, credit and asset information. A conventional process relies on policy rules and manual assessment. Although domain expertise remains essential, manual review can be slow and inconsistent when application volumes increase. Machine learning can support this workflow by learning patterns from historical labelled cases and providing a repeatable probability-based assessment.

Loan approval prediction is a binary classification task. Each observation represents an applicant and the target indicates either Approved or Rejected. The useful system is not merely a trained algorithm: it must include reproducible preprocessing, evaluation on unseen data, model selection, saved transformation objects and a stable interface for inference. CrediShield AI was designed around this full lifecycle.

The application uses eleven predictors: number of dependents, education, self-employment status, annual income, requested loan amount, loan term, CIBIL score and four asset-value fields. These features combine repayment capacity, credit history, requested exposure and indicators of financial strength.

1.1.1 Motivation

- Reduce turnaround time for preliminary screening.
- Apply the same learned decision logic to every submitted record.
- Compare interpretable and ensemble approaches rather than relying on one algorithm.
- Expose model confidence and probability estimates in a usable interface.
- Demonstrate a reproducible path from dataset to deployed prediction.

1.1.2 Project Scope

Included	Outside the Current Scope
Data preparation and supervised classification	Final regulated lending decisions
Four-model performance comparison	Credit bureau integration
Saved model and scaler	Production identity and fraud checks
Flask dashboard and prediction API	Continuous monitoring infrastructure

1.2 Problem Statement

The project addresses the need for a fast, consistent and measurable method of estimating loan approval outcomes from structured application data. The system must transform raw values exactly as they were transformed during training, compare alternative classifiers using suitable metrics, retain the best model, and serve predictions through an interface that can be used without directly executing machine learning code.

A weak solution may report high accuracy while missing many cases from one class, leak test information into preprocessing, or use different feature ordering at inference. The engineering problem therefore includes safeguards beyond raw predictive performance: stratified splitting, training-only scaler fitting, persisted feature metadata, exact column ordering and explicit error handling.

1.2.1 Research Questions

- Which of Logistic Regression, Decision Tree, Random Forest and XGBoost performs best on the held-out data?
- Does an ensemble model provide a useful improvement over a linear baseline?
- Can the selected pipeline be packaged for real-time prediction while preserving preprocessing consistency?
- Which applicant variables contribute most strongly to the selected model's decisions?

1.2.2 Constraints and Assumptions

The project assumes the supplied labels are correct and the development data broadly represents future applicants. Monetary values and loan terms must follow the same units used in training. Missing-value imputation is not implemented in the training module, so the dataset is assumed to be complete after cleaning. The local training entry point contains a machine-specific dataset path, which should be converted into a configuration or command-line argument for portability.

1.2.3 Success Criteria

Criterion	Evidence
Predictive quality	High F1-score and accuracy on a stratified holdout set
Reproducibility	Fixed random state and persisted scaler/model
Comparability	Same split and metrics for all candidate models
Usability	Browser form and JSON prediction endpoint
Traceability	Saved comparison metrics and diagnostic plots

1.3 Objectives

1.3.1 Primary Objective

To design, implement and evaluate an end-to-end machine learning system that predicts loan approval status from structured applicant information and makes the selected model available through a web application.

1.3.2 Specific Objectives

- Clean column names and categorical values.
- Cap extreme numeric values using the IQR rule.
- Encode categorical features and the binary target consistently.
- Create an 80:20 stratified train–test split with a reproducible random seed.
- Fit a StandardScaler only on training data and save it for inference.

- Train Logistic Regression, Decision Tree, Random Forest and XGBoost classifiers.
- Evaluate accuracy, precision, recall, F1-score and ROC-AUC.
- Select the model with the highest F1-score and persist its metadata.
- Generate a confusion matrix and feature-importance visualisation.
- Deploy predictions through Flask with form and JSON input support.

1.3.3 Expected Deliverables

Deliverable	Repository Artifact
Preprocessing module	src/data_preprocessing.py
Training and evaluation module	src/train_models.py
Inference module	src/predict.py
Web application	app/main.py and app/templates/index.html
Serialized assets	models/best_model.pkl and scaler.pkl
Evaluation outputs	model_comparison_metrics.csv and PNG plots

Chapter 2: Literature Review

2.1 Existing Approaches

Credit decision systems commonly combine scorecards, policy rules and statistical or machine learning models. Traditional logistic regression remains widely useful because its coefficients can be interpreted as directional relationships and its probability output is easy to calibrate. However, a linear decision surface may not capture interactions among income, loan amount, credit score, assets and loan duration.

Decision trees create human-readable rules and naturally represent nonlinear thresholds, but a single tree can be unstable and prone to overfitting. Random Forest reduces this variance by averaging many trees trained on bootstrapped samples and random feature subsets. Gradient boosting methods such as XGBoost construct trees sequentially to correct previous errors and often deliver excellent predictive performance on structured tabular data.

2.1.1 Evaluation Measures

Accuracy measures the overall proportion of correct classifications. Precision measures how often predicted members of the positive class are correct, whereas recall measures how many actual positive cases are detected. F1-score balances precision and recall through their harmonic mean. ROC-AUC evaluates ranking quality across classification thresholds. Because class labels in this implementation map Rejected to 1, the reported precision, recall and F1 values primarily describe rejection detection.

Metric	Interpretation in This Project
Accuracy	Overall correct approval/rejection predictions
Precision	Reliability of predicted rejections

Recall	Share of actual rejections detected
F1-score	Balanced rejection precision and recall
ROC-AUC	Ability to rank the two classes across thresholds

2.2 Research Studies and Project Positioning

Applied studies of credit scoring consistently report that ensemble tree methods are strong candidates for structured financial data because they handle nonlinearities and feature interactions without requiring extensive manual transformations. At the same time, responsible credit modelling literature emphasises interpretability, representative data, fairness testing, stability and governance. A numerically strong offline model is therefore only one layer of a trustworthy lending system.

This project adopts a comparative experimental design: a linear baseline, a single nonlinear tree, a bagged ensemble and a boosted ensemble are trained under the same preprocessing and split conditions. This makes the performance differences more meaningful than results obtained from separate datasets or inconsistent evaluation procedures.

2.2.1 Identified Gap

Many educational implementations stop after printing a score in a notebook. CrediShield AI addresses the implementation gap by persisting the scaler and model, storing the expected feature list, creating diagnostic graphics and exposing a reusable prediction function and HTTP endpoint. This turns the exercise into a small but complete machine learning product.

2.2.2 Conceptual Framework

- Input layer: applicant, credit, loan and asset variables.
- Processing layer: cleaning, clipping, encoding, splitting and scaling.
- Learning layer: candidate model training and holdout evaluation. • Selection layer: highest F1-score determines the saved estimator.
- Delivery layer: Flask UI/API returns class probabilities and prediction.
- Governance layer: fairness, explainability, monitoring and human oversight are future requirements.

Chapter 3: Methodology

3.1 Dataset Description

The project documentation identifies the Kaggle Loan Approval Prediction Dataset, containing 4,269 rows and 13 original columns. The identifier loan_id is removed, loan_status is the target, and eleven fields are retained as predictors.

Variable	Type	Description
no_of_dependents	Numeric	Number of dependents

education	Categorical	Graduate / Not Graduate
self_employed	Categorical	Yes / No
income_annum	Numeric	Annual income
loan_amount	Numeric	Requested amount
loan_term	Numeric	Loan duration
cibil_score	Numeric	Credit score
residential_assets_value	Numeric	Residential assets
commercial_assets_value	Numeric	Commercial assets
luxury_assets_value	Numeric	Luxury assets
bank_asset_value	Numeric	Bank assets
loan_status	Target	Approved / Rejected

3.1.1 Target Encoding

Approved is encoded as 0 and Rejected as 1. This detail is important when interpreting metrics and class probabilities: probability_approved uses index 0, while probability_rejected uses index 1.

3.2 Data Preprocessing

The preprocessing function loads the CSV with pandas, removes leading and trailing whitespace, clips numeric outliers, removes the identifier, maps categories, creates a stratified split, scales features and optionally saves the fitted scaler.

Step	Implementation	Purpose
Cleaning	Strip column and string whitespace	Prevent category mismatches
Outlier treatment	Clip beyond Q1−1.5×IQR and Q3+1.5×IQR	Limit extreme influence
Identifier removal	Drop loan_id	Avoid learning arbitrary identifiers
Encoding	Map two categorical predictors and target	Create numeric model inputs
Splitting	80:20, stratified, random_state=42	Preserve class ratio and reproducibility
Scaling	StandardScaler fitted on X_train	Avoid test leakage
Persistence	Save scaler.pkl	Reuse identical transformation at inference

3.2.1 IQR Capping

For each numeric variable, the first and third quartiles define the interquartile range. Values below Q1 − 1.5×IQR are replaced with the lower bound and values above Q3 + 1.5×IQR are replaced with the upper bound. Unlike row deletion, clipping retains all observations while limiting extreme magnitudes.

3.2.2 Data Leakage Control

The scaler is fitted only after the split and only on X_train. X_test is transformed using the already-fitted training scaler. The persisted object is later loaded by the prediction module, preventing training-serving skew.

3.3 Algorithms Used

Algorithm	Role	Key Property
Logistic Regression	Linear baseline	Probabilistic, simple and interpretable
Decision Tree	Nonlinear baseline	Rule-based splits; max_depth=6
Random Forest	Bagging ensemble	Averages multiple decorrelated trees
XGBoost	Boosting ensemble	Sequential error correction

3.3.1 Training Protocol

Every model receives the same scaled training and test matrices. Logistic Regression uses max_iter=1000 and random_state=42. Decision Tree limits depth to six. Random Forest and XGBoost use random_state=42; XGBoost uses log-loss evaluation. For each estimator, predictions, probabilities and five performance metrics are computed.

3.3.2 Model Selection Rule

The implementation selects the maximum F1-score. This criterion was preferred over accuracy alone because it penalises a model that performs poorly on either precision or recall. The selected estimator, its name, input feature order and metric values are stored together in best_model.pkl.

3.3.3 Experimental Reproducibility

- Fixed random_state=42 for split and estimators that expose it.
- Common holdout partition for all models.
- Same preprocessing and feature order.
- Machine-readable metrics saved to CSV.
- Diagnostic images generated automatically after training.

Chapter 4: Implementation

4.1 Exploratory Data Analysis (EDA)

EDA should verify data types, category spelling, target balance, missing values and numeric distributions before training. The repository's training code directly performs cleaning and produces post-training diagnostics; the dataset summary in the README provides the observation count and feature definitions.

4.1.1 Recommended EDA Checks

- Inspect shape, duplicate rows and null counts.
- Compare Approved and Rejected class frequencies.
- Plot distributions of income, requested amount, loan term and CIBIL score.
- Examine correlations among monetary asset variables.

- Compare credit score and loan-to-income patterns by target class.
- Document changes caused by IQR clipping.

4.1.2 Interpretation Considerations

Strong correlations among asset fields may create redundant predictive signals. A high-performing tree ensemble can still exploit these variables, but feature importance should not be interpreted as causal impact. Similarly, a relationship between CIBIL score and approval reflects historical labels and policy, not a universal rule.

4.1.3 Data Quality Risks

Risk	Potential Effect	Mitigation
Whitespace in categories	Failed encoding and missing values	String stripping
Extreme monetary values	Distorted scale and unstable fit	IQR clipping
Unit inconsistency	Invalid predictions	Input validation and documentation
Population shift	Performance decay	Monitoring and retraining

4.2 Feature Engineering

The implemented pipeline deliberately uses limited feature engineering: categorical mapping, numeric clipping and scaling. This keeps the feature space traceable and reduces the chance of training-serving inconsistency. The original variables are meaningful to users and can be collected directly from the web form.

4.2.1 Implemented Transformations

Feature Group	Transformation
education	Graduate=0; Not Graduate=1
self_employed	No=0; Yes=1
loan_status	Approved=0; Rejected=1
numeric predictors	IQR capping and standardisation
loan_id	Removed

4.2.2 Potential Derived Features

Future versions could test loan-to-income ratio, total asset value, debt-service proxies, asset-to-loan ratio and interactions between credit score and requested exposure. Such features should be defined in a shared transformer so that identical logic runs during training and inference.

- Loan-to-income ratio = loan_amount / income_annum.
- Total assets = sum of four asset-value fields.
- Asset coverage = total assets / loan_amount.
- Income per dependent = income_annum / (dependents + 1).

4.2.3 Validation Requirement

Derived variables should be accepted only when cross-validation or a separate validation set demonstrates stable improvement. They must also be checked for zero denominators, extreme ratios, leakage and fairness impact.

4.3 Model Development

The training module orchestrates preprocessing, candidate fitting, metric calculation, best-model selection and artifact generation. Paths are resolved relative to the source file for model and static image outputs, improving reliability when the script is launched from different working directories.

4.3.1 End-to-End Workflow

Stage	Input	Output
1. Load	CSV dataset	pandas DataFrame
2. Prepare	Raw fields	Encoded train/test sets
3. Train	X_train, y_train	Four fitted classifiers
4. Evaluate	X_test, y_test	Metrics and confusion matrices
5. Select	F1-scores	Random Forest
6. Persist	Best estimator + scaler	PKL artifacts
7. Serve	Web/JSON request	Prediction and probabilities

4.3.2 Artifact Design

best_model.pkl is a metadata dictionary containing the estimator, model name, expected feature order and selected metrics. scaler.pkl stores the fitted StandardScaler. Keeping feature names with the model allows predict.py to construct inputs in exactly the training order.

4.4 Web Application and API

The Flask application provides a home route and a POST /predict route. The home page checks whether the model exists and displays model information. The prediction route accepts either HTML form data or JSON, converts values into the required types, invokes the shared inference function and returns a JSON result.

4.4.1 Prediction Contract

Request Category	Fields
Applicant	no_of_dependents, education, self_employed
Loan	loan_amount, loan_term
Financial	income_annum, four asset values
Credit	cibil_score

The response includes success, prediction, confidence, probability_approved, probability_rejected and model_used. File-not-found errors return HTTP 400 with a training instruction; unexpected errors return HTTP 500 with diagnostic details.

4.4.2 Deployment Strengths

- One inference function is reused by the web layer.
- Model and scaler paths are derived from the project structure.
- Both form and JSON clients are supported.
- Class probabilities increase transparency for users.
- The interface checks model availability before use.

4.4.3 Production Hardening

- Validate allowed ranges and reject impossible monetary or credit values.
- Disable Flask debug mode in production.
- Avoid returning raw exception details to public clients.
- Add authentication, HTTPS, audit logging and rate limiting.
- Containerise deployment and use Gunicorn or another production WSGI server.

Chapter 5: Results and Discussion

5.1 Performance Metrics

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	92.27%	91.59%	87.62%	89.56%	97.45%
Decision Tree	97.19%	99.02%	93.50%	96.18%	99.37%
Random Forest	98.36%	99.36%	96.28%	97.80%	99.79%
XGBoost	98.01%	98.42%	96.28%	97.34%	99.86%

Random Forest achieved the best F1-score and accuracy. XGBoost produced the highest ROC-AUC, but its F1-score was slightly lower. Since the implementation's explicit selection rule is maximum F1-score, Random Forest was correctly retained as the deployment model.

5.1.1 Absolute Improvements

- Random Forest improves accuracy over Logistic Regression by approximately 6.09 percentage points.
- Random Forest improves F1-score over Logistic Regression by approximately 8.24 percentage points.

- Random Forest and XGBoost have equal reported recall (96.28%), while Random Forest has higher precision.
- The Decision Tree is competitive but remains below both ensembles on accuracy and F1-score.

5.1.2 Important Metric Caveat

Because Rejected is encoded as class 1, the precision, recall and F1 values generated with scikit-learn defaults describe rejection classification. For a full operational assessment, class-specific metrics, macro averages, calibration and business-cost analysis should also be reported.

5.2 Diagnostic Results

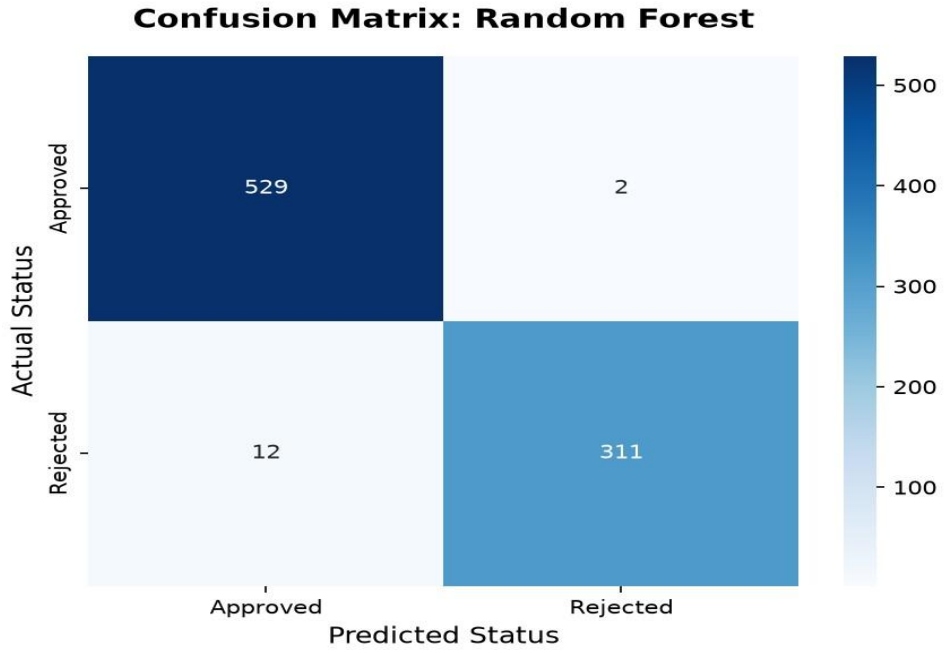

Figure 5.1. Confusion matrix for the selected Random Forest model.

The confusion matrix summarises correct and incorrect outcomes by actual and predicted class. Its strongly populated diagonal is consistent with the 98.36% overall accuracy. The remaining off-diagonal cases represent false approvals and false rejections; in a lending context these error types have different costs and should be weighted explicitly.

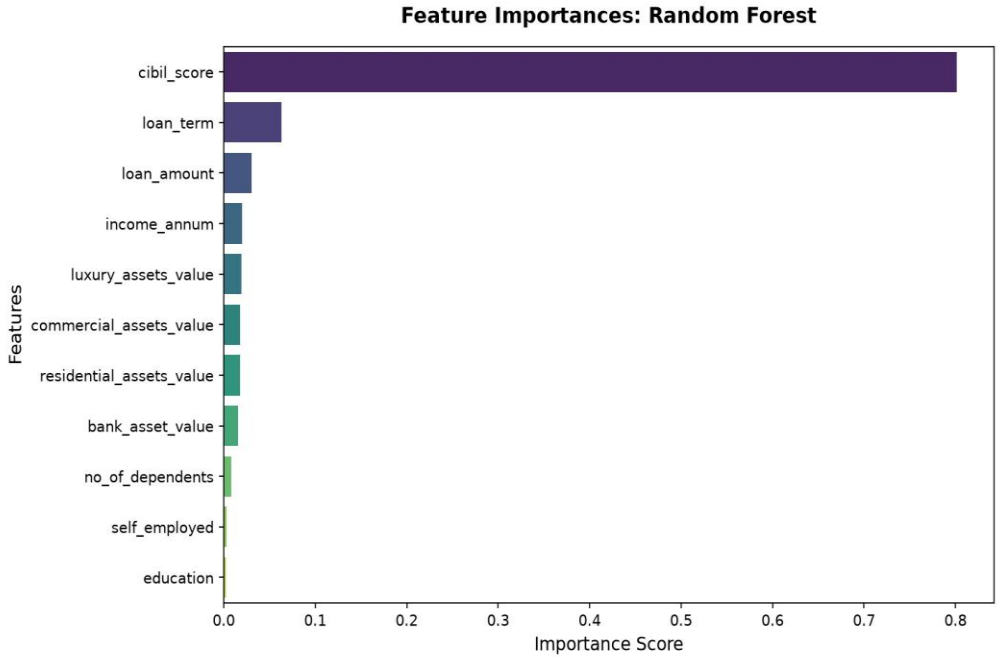

Figure 5.2. Feature importance produced by the selected model.

Feature importance identifies variables frequently useful for the forest's splits. It supports model inspection but does not prove causality or fairness. Case-level explanations such as SHAP values would be preferable when explaining an individual decision.

5.3 Comparative Analysis and Discussion

Model	Strength	Observed Limitation
Logistic Regression	Clear baseline and probability model	Lowest recall and F1
Decision Tree	Readable nonlinear rules	Lower generalisation than ensembles
Random Forest	Best accuracy, precision and F1	Less transparent than one tree
XGBoost	Highest ROC-AUC	Slightly lower F1 than Random Forest

The results indicate meaningful nonlinear structure in the dataset. Both ensemble methods substantially outperform Logistic Regression, and the constrained Decision Tree also performs strongly. Random Forest offers the best balance under the selected criterion, while XGBoost may be attractive if threshold-independent ranking is prioritised.

5.3.1 Practical Interpretation

A high test score demonstrates that the model reproduces patterns in this holdout sample; it does not by itself prove suitability for live lending. Deployment should define the cost of incorrectly approving a risky applicant versus incorrectly rejecting a creditworthy applicant. The classification threshold can then be selected using expected cost, policy constraints and risk appetite rather than the default 0.5 alone.

5.3.2 Threats to Validity

- A single train–test split may overstate or understate generalisation.
- The dataset may not represent a particular lender's future population.
- Historical labels may encode past policy or bias.
- No subgroup fairness analysis is reported.
- No probability calibration or temporal validation is included.
- The dataset file is external to the repository, limiting immediate reproducibility.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

CrediShield AI successfully demonstrates a complete machine learning workflow for loan approval prediction. The project converts raw tabular data into reproducible training and test sets, compares four classification algorithms, selects a model using F1-score, stores the trained artifacts and serves real-time predictions through Flask.

Random Forest was selected with 98.36% accuracy, 99.36% precision, 96.28% recall, 97.80% F1-score and 99.79% ROC-AUC. The close performance of XGBoost confirms that ensemble tree methods are well suited to this dataset. The modular separation among preprocessing, training, prediction and web serving makes the implementation understandable and extensible.

6.2 Future Scope

- Use repeated stratified cross-validation and temporal validation.
- Add hyperparameter optimisation with a protected validation strategy.
- Implement a unified scikit-learn Pipeline or ColumnTransformer.
- Add missing-value handling and strict schema/range validation.
- Evaluate calibration and choose thresholds from business costs.
- Introduce SHAP-based global and local explanations.
- Measure fairness across legally appropriate, consented groups.
- Track prediction drift, outcome drift and metric degradation.
- Store model versions and training metadata in a model registry.
- Add automated tests, CI/CD, containerisation and secure production hosting.

6.3 Final Recommendation

The current system is appropriate as an academic prototype and decision-support demonstration. Before operational use, it should undergo independent validation, fairness and compliance review, security hardening, monitoring design and integration with human underwriting controls.

References

1. Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD Conference.
3. Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. Machine Learning, 20, 273–297.
4. Fawcett, T. (2006). An Introduction to ROC Analysis. Pattern Recognition Letters, 27(8), 861–874.
5. Fisher, R. A. (1936). The Use of Multiple Measurements in Taxonomic Problems. Annals of Eugenics, 7(2), 179–188.
6. Kaggle. Loan Approval Prediction Dataset. https://www.kaggle.com/datasets/architsharma01/loan-approval-prediction-dataset
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
8. Python Software Foundation. Python Documentation. https://docs.python.org/
9. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
10. Project repository source files: data_preprocessing.py, train_models.py, predict.py and app/main.py (accessed July 2026).

Appendix A: Reproduction Guide

- Install packages from requirements.txt.
- Place the dataset at the configured path or update csv_file in src/train_models.py.
- Run: python src/train_models.py.
- Confirm the model, scaler, CSV metrics and figures are generated.
- Run: python app/main.py.
- Open http://localhost:5000 and submit a test application.